

SmartCity Mobility Mega Assessment (Q1000)

How to run

```
```bash
```

```
python3 -m venv venv
```

```
source venv/bin/activate
```

```
pip install -r requirements.txt
```

```
python3 test_runner.py
```

```
```
```

This will:

- run the tests defined in `test\_config.json`
- load your functions from `solution.py`
- print `[PASS]` / `[FAIL]` per test case
- write results to `test\_report.log`

Dataset

CSV files are provided in `data/`:

- `data/mobility\_trips.csv`
- `data/vehicles.csv`
- `data/users.csv` (for your own practice; tests use inline profiles/plans too)

`mobility\_trips.csv` columns:

- trip_id (string)
- user_id (string)
- vehicle_id (string)

- zone (string)
- trip_date (string `yyyy-MM-dd`)
- start_ts (string `yyyy-MM-dd HH:mm:ss`)
- end_ts (string `yyyy-MM-dd HH:mm:ss`)
- distance_km (double)
- fare (double)
- discount (double)
- payment_method (string)
- vehicle_type (string)
- speed_kmh (double)
- harsh_brake_count (int)
- status (string: Completed/Cancelled)

Objective

Implement **40 pure PySpark functions** in `solution.py`.

Important testing note (no dependency)

Each function is tested **independently**.

For functions that normally depend on upstream cleaning/aggregation, the tests pass a **precomputed DataFrame** fixture.

A) Core Trip Analytics (1–20)

1) define_trip_schema() -> StructType

Goal: Explicit schema for mobility_trips.csv with all nullable fields.

Flow:

Create a StructType with fields in the same order as CSV columns.

Keep:

trip_date, start_ts, end_ts as string.

numeric fields as double/int as specified.

Return schema.

2) load_trips(spark, path, schema) -> DataFrame

Goal: Load CSV with header and your schema.

Flow:

Use Spark CSV reader with:

header enabled

schema applied

Return DataFrame.

3) parse_trip_date(df) -> DataFrame

Goal: Convert trip_date string to DateType.

Flow:

Replace/overwrite trip_date column using date parsing.

Return updated DF.

4) add_trip_duration_min(df) -> DataFrame

Goal: Add integer trip duration in minutes.

Flow:

Convert start_ts and end_ts to epoch seconds.

Compute (end - start) / 60.

Round as required by spec (or floor if tests expect it—spec says integer minutes; most consistent is rounding to nearest then cast).

Add new column: trip_duration_min as int.

Return DF.

5) add_cost_per_km(df) -> DataFrame

Goal: Add cost_per_km = fare / distance_km.

Flow:

Create new column cost_per_km as double.

(Optionally handle distance_km = 0 → null/inf; tests usually avoid zero.)

Return DF.

6) filter_peak_hour_trips(df) -> DataFrame

Goal: Keep trips starting in peak hours: 07–10 OR 17–20 inclusive.

Flow:

Extract hour from start_ts.

Filter where hour is in either range (inclusive).

Return filtered DF.

7) top_n_users_by_distance(df, n) -> DataFrame

Goal: Top users by total distance.

Flow:

Group by user_id.

Aggregate sum of distance_km as total_distance_km.

Order:

total_distance_km desc

user_id asc

Limit to n.

Return DF.

8) avg_speed_by_zone(df) -> DataFrame

Goal: Average speed per zone.

Flow:

Group by zone.

Compute average of speed_kmh as avg_speed_kmh.

Return DF.

9) most_common_vehicle_type(df) -> str

Goal: Most frequent vehicle_type (tie-break asc).

Flow:

Group by vehicle_type, count rows.

Order by:

count desc

vehicle_type asc

Take first row.

If DF empty → return "".

Return string vehicle_type.

10) cancellation_rate_by_zone(df) -> DataFrame

Goal: cancelled/total per zone.

Flow:

Group by zone.

Compute:

total count

cancelled count using conditional sum where status == "Cancelled"

Compute cancellation_rate = cancelled / total as double.

Select only: zone, cancellation_rate.

Return DF.

11) high_duration_trips(df, threshold_min) -> DataFrame

Goal: duration > threshold.

Flow:

If trip_duration_min missing:

compute it (same logic as function 4)

Filter where trip_duration_min strictly greater than threshold.

Return DF.

12) count_active_vehicles_by_zone(vehicles_df) -> DataFrame

Goal: Count active vehicles per home zone.

Flow:

Filter vehicles where status == "Active".

Group by home_zone.

Count distinct vehicle_id as active_vehicles.

Rename home_zone → zone.

Return DF.

13) daily_net_revenue_trend(df) -> DataFrame

Goal: Daily sum of (fare - discount).

Flow:

Create net_revenue = fare - discount (double).

Group by trip_date.

Sum net_revenue as daily_net_revenue.

Order by trip_date asc.

Return DF.

14) top_vehicle_by_net_revenue(df) -> Tuple[str, float]

Goal: Vehicle with highest total net revenue.

Flow:

Compute net_revenue per row (fare - discount).

Group by vehicle_id, sum net_revenue as total_net_revenue.

Order by:

total_net_revenue desc

vehicle_id asc

Take first:

If empty → return ("", 0.0)

Else return (vehicle_id, total_net_revenue) with float conversion.

15) list_zones(df) -> List[str]

Goal: Unique zones sorted ascending.

Flow:

Select distinct zones.

Collect to driver as list.

Convert values to strings (safe).

Sort in Python.

Return list.

16) trips_in_date_range(df, start, end) -> DataFrame

Goal: trip_date between start and end inclusive.

Flow:

Ensure comparison uses DateType values:

Convert start/end strings into date literals.

Filter with inclusive between.

Return DF.

17) flag_safety_risk(df, speed_threshold, brake_threshold) -> DataFrame

Goal: Add boolean is_risky.

Flow:

is_risky = (speed_kmh > threshold) OR (harsh_brake_count > threshold)

Add column is_risky boolean.

Return DF.

18) top_n_risky_vehicles(df, n) -> DataFrame

Goal: Count risky trips per vehicle.

Flow:

Filter where is_risky == True.

Group by vehicle_id.

Count rows as risky_count.

Order:

risky_count desc

vehicle_id asc

Limit n.

Return DF.

19) avg_fare_by_payment_method(df) -> DataFrame

Goal: Avg fare per payment method.

Flow:

Group by payment_method.

Aggregate avg(fare) as avg_fare.

Return DF.

20) get_longest_trip(df) -> Tuple[str, int]

Goal: Longest by duration, tie trip_id asc.

Flow:

Ensure duration exists (compute if missing).

Order:

trip_duration_min desc

trip_id asc

Select trip_id and duration.

If empty → ("", 0)

Return (trip_id, trip_duration_min).

B) Inline join + enrichment (21–25)

21) define_user_profile_schema() -> StructType

Goal: Schema for inline profiles.

Flow:

Create StructType with:

user_id, first_name, last_name, full_name, plan_id (all string, nullable)

Return schema.

22) define_vehicle_schema() -> StructType

Goal: Schema for inline plans.

Flow:

Create StructType with:

plan_id, plan_name (string, nullable)

Return schema.

```
"""Create inline profile and plan lookup DataFrames and return (profiles_df,
plans_df)."""
```

```
profiles_rows = [
    ("U1", "Aarav", "Sharma", None, "P2"),
    ("U2", "Diya", "Iyer", "Diya Iyer", "P1"),
    ("U3", "Kabir", "Singh", None, "P1"),
    ("U4", "Meera", "Nair", None, "P2"),
]

plans_rows = [
    ("P1", "Standard"),
    ("P2", "Premium"),
]
```

23) load_inline_profiles_and_plans(...) -> Tuple[DataFrame, DataFrame]

Goal: Create two small lookup DataFrames.

Flow:

Define rows as Python lists of tuples for:

profiles

plans

Use `spark.createDataFrame(rows, schema=...)` for each.

Return (profiles_df, plans_df).

24) `join_profiles_with_plans(profiles_df, plans_df) -> DataFrame`

Goal: Left join profiles → plans on plan_id.

Flow:

Left join on plan_id.

Select output columns exactly:

user_id, first_name, last_name, full_name, plan_id, plan_name

Return DF.

25) `enrich_full_name(df) -> DataFrame`

Goal: Fill missing/blank full_name from first + last.

Flow:

Detect when full_name is:

null OR trimmed length == 0

Replace with concatenation of first_name and last_name with a space.

Otherwise keep existing.

Return DF.

C) Unix timestamp conversions (26–27)

26) add_start_epoch_seconds(df) -> DataFrame

Goal: Add epoch seconds from start_ts.

Flow:

Convert start_ts string to epoch seconds.

Cast to long.

Add start_epoch_seconds.

Return DF.

27) add_start_ts_from_epoch(df) -> DataFrame

Goal: Convert epoch seconds back to timestamp.

Flow:

Take start_epoch_seconds.

Convert from unix time to timestamp.

Add column start_ts_from_epoch as TimestampType.

Return DF.

D) Additional analytics (28–40)

28) revenue_share_by_vehicle_type(metrics_df) -> List[str]

Input: columns vehicle_type, rev

Goal: Return vehicle_types ordered by revenue share desc.

Flow:

Group by vehicle_type, sum rev as total_rev.

Compute grand total (sum of total_rev).

Edge case:

if grand is null/0 → return []

Add share = total_rev / grand_total.

Order:

share desc

vehicle_type asc (tie-break)

Collect ordered vehicle_type into Python list and return.

29) busiest_zone_by_trips(df) -> Tuple[str, int]

Goal: Zone with max trip count.

Flow:

Group by zone, count rows.

Order:

count desc

zone asc

Take first:

empty -> ("", 0)

Return (zone, count).

30) completed_trips_by_zone(df) -> DataFrame

Goal: Count completed trips per zone.

Flow:

Filter status == "Completed".

Group by zone, count as completed_trips.

Order:

completed_trips desc

zone asc

Return DF.

31) add_utilization_score(df) -> DataFrame

Goal: utilization_score = distance_km * trip_duration_min.

Flow:

Ensure trip_duration_min exists (compute if missing).

Create utilization_score as double.

Return DF.

32) top_n_users_by_utilization(df, n) -> DataFrame

Goal: Top users by total utilization.

Flow:

Ensure utilization_score exists (compute if missing).

Group by user_id, sum utilization_score as total_utilization.

Order:

total_utilization desc

user_id asc

Limit n.

Return DF.

33) median_duration_by_zone(df) -> DataFrame

Goal: Median trip duration per zone.

Flow:

Ensure trip_duration_min exists.

Group by zone.

Use approximate median aggregation (percentile_approx at 0.5).

Alias column median_duration_min.

Return DF.

34) minmax_normalize_fare(df) -> DataFrame

Goal: $\text{fare_norm} = (\text{fare} - \text{min}) / (\text{max} - \text{min})$.

Flow:

Compute global min(fare) and max(fare) by aggregating once to driver.

Edge case:

if min/max missing OR $\text{max} == \text{min}$ → add $\text{fare_norm} = 0.0$ for all rows.

Else compute normalized value per row.

Return DF.

35) detect_outlier_fares(df) -> DataFrame

Goal: is_outlier if $\text{fare} > \text{mean} + 2 * \text{stddev_pop}$.

Flow:

Aggregate global mean and stddev_pop of fare (collect once).

If mean or stddev missing → set is_outlier False.

Else compute threshold and add boolean column is_outlier .

Return DF.

36) monthly_net_revenue_by_zone(df) -> DataFrame

Goal: Sum net revenue by month+zone.

Flow:

Ensure trip_date is DateType (tests may already provide it; don't force parse unless needed).

Compute net_revenue = fare - discount.

Compute month string formatted yyyy-MM.

Group by (month, zone), sum net_revenue as monthly_net_revenue.

Order by month asc, zone asc.

Return DF.

37) weekday_peak_trip_counts(df) -> DataFrame

Goal: Peak trips count per weekday label.

Flow:

Get peak-hour trips (reuse same filtering logic as function 6).

Extract weekday label from trip_date (e.g., Mon/Tue...).

Group by weekday, count as peak_trips.

Return DF.

38) pivot_payment_counts_by_zone(df) -> DataFrame

Goal: Pivot counts by payment method for each zone.

Flow:

Group by zone.

Pivot on payment_method.

Aggregate count.

Fill nulls with 0 (so zones missing a method get 0).

Return pivoted DF.

39) calculate_net_revenue(df) -> DataFrame

Goal: Add net_revenue = fare - discount.

Flow:

Add/overwrite net_revenue as double.

Return DF.

40) flag_service_due(vehicles_df, due_days) -> DataFrame

Goal: is_service_due based on last_service_date.

Flow:

Compute days since last service: datediff(current_date, last_service_date).

is_service_due = diff > due_days.

Add boolean column is_service_due.

Return DF.

Practical tips to avoid FAILs

Always alias exactly as specified (total_distance_km, avg_speed_kmh, daily_net_revenue, etc.).

Apply tie-break sorting even when it feels unnecessary.

For tuple returns, always handle empty input safely.

Don't mutate input variables in surprising ways—return new DF each time.

from __future__ import annotations

from typing import List, Tuple

```

from pyspark.sql import SparkSession, DataFrame
from pyspark.sql import functions as F
from pyspark.sql.types import (
    StructType,
    StructField,
    StringType,
    DoubleType,
    IntegerType,
    TimestampType,
    DateType,
    LongType,
)

# =====

# A) Core Trip Analytics

# =====

def define_trip_schema() -> StructType:
    pass

def load_trips(spark: SparkSession, path: str, schema: StructType) -> DataFrame:
    pass

def parse_trip_date(df: DataFrame) -> DataFrame:
    pass

def add_trip_duration_min(df: DataFrame) -> DataFrame:
    pass

```

```
def add_cost_per_km(df: DataFrame) -> DataFrame:
```

```
    pass
```

```
def filter_peak_hour_trips(df: DataFrame) -> DataFrame:
```

```
    pass
```

```
def top_n_users_by_distance(df: DataFrame, n: int) -> DataFrame:
```

```
    pass
```

```
def avg_speed_by_zone(df: DataFrame) -> DataFrame:
```

```
    pass
```

```
def most_common_vehicle_type(df: DataFrame) -> str:
```

```
    pass
```

```
def cancellation_rate_by_zone(df: DataFrame) -> DataFrame:
```

```
    pass
```

```
def high_duration_trips(df: DataFrame, threshold_min: int) -> DataFrame:
```

```
    pass
```

```
def count_active_vehicles_by_zone(df: DataFrame) -> DataFrame:
```

```
    pass
```

```
def daily_net_revenue_trend(df: DataFrame) -> DataFrame:
```

```
    pass
```

```
def top_vehicle_by_net_revenue(df: DataFrame) -> Tuple[str, float]:  
    pass
```

```
def list_zones(df: DataFrame) -> List[str]:  
    pass
```

```
def trips_in_date_range(df: DataFrame, start: str, end: str) -> DataFrame:  
    pass
```

```
def flag_safety_risk(  
    df: DataFrame, speed_threshold: float, brake_threshold: int  
) -> DataFrame:  
    pass
```

```
def top_n_risky_vehicles(df: DataFrame, n: int) -> DataFrame:  
    pass
```

```
def avg_fare_by_payment_method(df: DataFrame) -> DataFrame:  
    pass
```

```
def get_longest_trip(df: DataFrame) -> Tuple[str, int]:  
    pass
```

```
# =====
```

```
# B) Inline Join + Enrichment
```

```
# =====
```

```
def define_user_profile_schema() -> StructType:
```

```
pass
```

```
def define_vehicle_schema() -> StructType:
```

```
    pass
```

```
def load_inline_profiles_and_plans(
```

```
    spark: SparkSession,
```

```
    profile_schema: StructType,
```

```
    plan_schema: StructType,
```

```
) -> Tuple[DataFrame, DataFrame]:
```

```
    pass
```

```
def join_profiles_with_plans(
```

```
    profiles_df: DataFrame, plans_df: DataFrame
```

```
) -> DataFrame:
```

```
    pass
```

```
def enrich_full_name(profile_joined_df: DataFrame) -> DataFrame:
```

```
    pass
```

```
# =====
```

```
# C) Unix timestamp conversions
```

```
# =====
```

```
def add_start_epoch_seconds(df: DataFrame) -> DataFrame:
```

```
    pass
```

```
def add_start_ts_from_epoch(df: DataFrame) -> DataFrame:
```

pass

=====

D) Additional analytics

=====

def revenue_share_by_vehicle_type(df: DataFrame) -> List[str]:

pass

def busiest_zone_by_trips(df: DataFrame) -> Tuple[str, int]:

pass

def completed_trips_by_zone(df: DataFrame) -> DataFrame:

pass

def add_utilization_score(df: DataFrame) -> DataFrame:

pass

def top_n_users_by_utilization(df: DataFrame, n: int) -> DataFrame:

pass

def median_duration_by_zone(df: DataFrame) -> DataFrame:

pass

def minmax_normalize_fare(df: DataFrame) -> DataFrame:

pass

def detect_outlier_fares(df: DataFrame) -> DataFrame:

```
pass
```

```
def monthly_net_revenue_by_zone(df: DataFrame) -> DataFrame:
```

```
    pass
```

```
def weekday_peak_trip_counts(df: DataFrame) -> DataFrame:
```

```
    pass
```

```
def pivot_payment_counts_by_zone(df: DataFrame) -> DataFrame:
```

```
    pass
```

```
def calculate_net_revenue(df: DataFrame) -> DataFrame:
```

```
    pass
```

```
def flag_service_due(df: DataFrame, due_days: int) -> DataFrame:
```

```
    pass
```

Clinic Appointment Wait-Time Assessment (Q777)

-
- `## How to run`
- `` `` ` bash`
- `python3 -m venv venv`
- `source venv/bin/activate`
- `pip install -r requirements.txt`
- `python3 test_runner.py`
- `` `` ``
-
- This will:
- - run the tests defined in ``test_config.json``
- - load your functions from ``solution.py``
- - print ``[PASS]` / `[FAIL]` per test case`
- - write results to ``test_report.log``
-
- `## Dataset`
- CSV file provided in ``data/appointments.csv``.
-
- Columns:
- - `appointment_id` (string)
- - `patient_id` (string)
- - `doctor` (string)
- - `department` (string)
- - `scheduled_time` (string ``yyyy-MM-dd HH:mm:ss``)
- - `actual_time` (string ``yyyy-MM-dd HH:mm:ss``)
- - `wait_reason` (string)
-

- ## Objective
- Implement **9 pure PySpark functions** in ``solution.py``.
-
- ### Important testing note (no dependency)
- Each function is tested **independently**.
- For functions that normally depend on upstream derived columns, the tests pass a **precomputed DataFrame fixture**.
-
- ---
-
- ## Function list and LLD (Implementation Flow)
-
- ### 1) `load_appointment_data(spark: SparkSession) -> DataFrame`
- - Load ``data/appointments.csv`` into a DataFrame using an explicit schema.
- - Use `header=True`.
- - Return a non-empty DataFrame with the exact columns listed in the dataset section.
-
- ### 2) `append_wait_minutes(df: DataFrame) -> DataFrame`
- - Compute wait time in minutes:
- - Convert ``scheduled_time`` and ``actual_time`` to epoch seconds.
- - `wait_minutes = floor((actual - scheduled) / 60)`
- - Add integer column ``wait_minutes``.
- - Return updated DataFrame.
-
- ### 3) `get_long_wait_appointments(df: DataFrame, threshold_minutes: int) -> DataFrame`
- - Compute wait in minutes (same logic as function 2) and add ``wait_minutes``.
- - Return only rows where ``wait_minutes > threshold_minutes`` (strictly greater).

-
- `### 4) most_delayed_doctor(df_with_wait: DataFrame) -> DataFrame`
- - Input already contains `wait\_minutes`.
- - Group by `doctor` and compute `sum(wait_minutes)` as `total\_wait`.
- - Order by:
- - `total_wait desc`
- - `doctor asc` (tie-break)
- - Return a DataFrame with exactly one row and columns: `doctor`, `total\_wait`.
-
- `### 5) long_wait_percentage(df_with_wait: DataFrame) -> float`
- - Input already contains `wait\_minutes`.
- - Define "long wait" as `wait\_minutes > 30`.
- - Compute percentage:
- - `(long_wait_count / total_count) * 100`
- - Return a Python float (e.g., 60.0).
-
- `### 6) most_delayed_appointment(df_with_wait: DataFrame) -> tuple[str, int]`
- - Input already contains `wait\_minutes`.
- - Find the appointment with the maximum `wait_minutes`.
- - Tie-break by `appointment\_id` asc.
- - Return `(appointment\_id, wait\_minutes)`.
- - If no rows: return `("", 0)`.
-
- `### 7) avg_wait_by_department(df_with_wait: DataFrame) -> DataFrame`
- - Input already contains `wait\_minutes`.
- - Group by `department` and compute `avg(wait_minutes)` as `avg\_wait\_minutes`.
- - Order by:

- - avg_wait_minutes desc
- - department asc
- - Return DataFrame with columns: `department`, `avg\_wait\_minutes`.
-
- `### 8) top_n_patients_by_wait(df_with_wait: DataFrame, n: int) -> DataFrame`
- - Input already contains `wait\_minutes`.
- - Group by `patient\_id` and sum(wait_minutes) as `total\_wait\_minutes`.
- - Order by:
- - total_wait_minutes desc
- - patient_id asc
- - Return top n rows.
-
- `### 9) wait_reason_counts(df: DataFrame) -> DataFrame`
- - Count appointments per `wait\_reason`.
- - Order by:
- - count desc
- - wait_reason asc
- - Return columns: `wait\_reason`, `reason\_count`.
-
- ---
-
- `## Notes`
- - Use only PySpark DataFrame APIs (no Pandas transformations).
- - Do not print, cache, or write files inside functions.
- - Ensure column names and return types match the specs exactly.
-
-